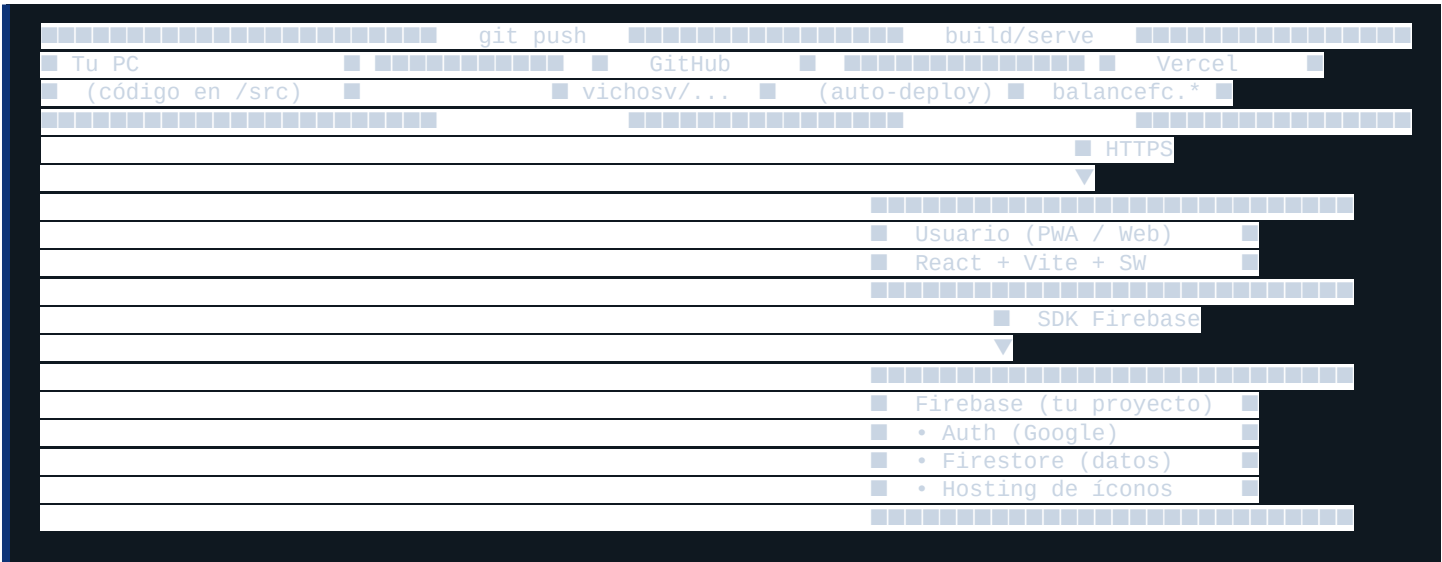


# BalanceFC — Arquitectura

App web para organizar fútbol de barrio: jugadores, equipos balanceados, ranking, convocatorias, tienda cosmética y apuestas internas con monedas ("goles").

## Stack en una imagen



## 1. Dónde vive el código y cómo llega al usuario

| Pieza         | Qué hace                                                                                                             | Dónde                                                                                             |
|---------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Código fuente | React + Vite. Todo lo visual y la lógica del cliente.                                                                | Tu PC + GitHub<br>(vichosv/BalanceFC-v2)                                                          |
| GitHub        | Repositorio. Cada <code>git push</code> dispara un deploy.                                                           | <a href="https://github.com/vichosv/BalanceFC-v2">github.com/vichosv/BalanceFC-v2</a>             |
| Vercel        | Hostea la app. Compila el proyecto con <code>npm run build</code> , sirve los archivos estáticos y la PWA por HTTPS. | URL pública<br>( <a href="https://balancefc-v2.vercel.app">balancefc-v2.vercel.app</a> o similar) |
| Firebase      | Backend gestionado. Auth + base de datos. Vive en tu cuenta de Google.                                               | Consola Firebase (proyecto v2)                                                                    |

### Flujo de despliegue:

- Editás archivos en `src/` (componentes React) o `firestore.rules`.
- `git commit` + `git push` a `main`.
- Vercel detecta el push, corre `npm install` + `npm run build`, y publica los assets nuevos.

4. Los usuarios reciben la versión nueva al recargar (la PWA además se auto-actualiza por el service worker).

**Las reglas de Firestore NO se despliegan con Vercel.** Las tenés que publicar manualmente en la consola de Firebase (Firestore → Rules → pegar `firestore.rules` → Publish).

## 2. El frontend (React + Vite)

Aplicación SPA. Entrada en `src/main.jsx` → `App.jsx` → `MainLayout.jsx`  
(decide qué pestaña mostrar según el estado `tab`).

|                                 |                                                                                                                               |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>src/</code>               |                                                                                                                               |
| <code>pages/</code>             | ← Una por pestaña ( <code>HomePage</code> , <code>MatchPage</code> , <code>HistoryPage</code> , etc.)                         |
| <code>components/</code>        | ← Reusables ( <code>PlayerCard</code> , <code>NavBar</code> , <code>EmptyState</code> , <code>modales</code> )                |
| <code>hooks/</code>             | ← Lógica de datos ( <code>useAuth</code> , <code>useMatches</code> , <code>useShopItems</code> , <code>useBets</code> , etc.) |
|                                 | Cada hook envuelve <code>queries/snapshots</code> de Firestore.                                                               |
| <code>utils/</code>             | ← Lógica pura ( <code>stats</code> , <code>logros</code> , <code>shop</code> , <code>challenges</code> , <code>teams</code> ) |
| <code>firebase/config.js</code> | ← Credenciales del proyecto Firebase ( <code>vars</code> de entorno)                                                          |

### Patrones clave:

- **Hooks de datos:** cada colección de Firestore tiene un hook con `onSnapshot` que mantiene los datos sincronizados en tiempo real (cualquier cambio en la DB se refleja al instante en todas las pantallas abiertas).
- **ctx:** en `MainLayout` se arma un objeto `ctx` con `user`, `isAdmin`, `players`, etc., y se pasa a cada página. Toggle "ver como jugador" se hace acá (anula `isAdmin` temporalmente).
- **PWA:** `vite-plugin-pwa` genera el service worker. El manifest (`favicon.svg` como ícono) hace que se pueda **instalar** en el celular y funcione **offline** (con la app cacheada; los datos siguen necesitando red).
- **Estilo:** Blue Lock — fondo hexagonal global + glassmorphism + animaciones (`fade-up`, `logo-glow`, `pulse-glow` en marcos).

## 3. Firebase: Auth + Firestore

**Firebase Auth** maneja el login con Google. Cada usuario logueado tiene un `uid` único.

**Firestore** es la base de datos NoSQL. Cada "colección" es una carpeta de documentos JSON. Estructura actual:

| Colección                  | Qué guarda                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>users/{uid}</code>   | Auth-side: <code>isAdmin</code> , <code>onboarded</code> , <code>email</code> . Se crea en el onboarding.                                                                                                                                                                                                                                                         |
| <code>players/{uid}</code> | Perfil de juego: <code>nickname</code> , <code>posición</code> , <code>foto</code> , <code>stats</code> ( <code>vel/tec/def/tir/sta/fis</code> ), <code>history</code> ( <code>matches/wins/goals/assists</code> ), <code>seasons</code> , <code>coins</code> , <code>inventory</code> , <code>equipped</code> , <code>statHistory</code> , <code>status</code> . |

|                                 |                                                                                                                                             |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>matches/{id}</code>       | Cada partido jugado: teamA/B/C, scoreA/B/C, playerStats, votes, videoUrl, mvpAwardedUid, gkAwardedUid.                                      |
| <code>seasons/{id}</code>       | Temporadas: nombre, status ( <code>active/closed</code> ).                                                                                  |
| <code>convocatorias/{id}</code> | Próximos partidos: fecha, hora, lugar, confirmados[]. Se auto-borran cuando pasa la hora.                                                   |
| <code>shopItems/{id}</code>     | Items custom de la tienda (los defaults están hardcodedos en <code>utils/shop.js</code> ). Pueden sobrescribir defaults usando el mismo id. |
| <code>bets/{id}</code>          | Apuestas: uid, tipo, monto, status ( <code>pending/won/lost</code> ).                                                                       |
| <code>moments/{id}</code>       | Momentos del partido (golazo/atajada/errado/blooper) con timestamp del video.                                                               |

Las **stats acumuladas** (history, seasons.) y *la evolución de atributos\** se actualizan automáticamente al guardar un partido — `useMatches.logMatch()` corre `buildStatUpdates`, `resolveBets` y `applyStatEvolution` en cascada.

## 4. Seguridad

`firestore.rules` es la única defensa: vive en Firebase y se ejecuta del lado del servidor. Sin reglas, cualquiera con el config del proyecto puede leer/escribir lo que sea.

Lo que **bloquean** las reglas actuales:

- Pumpear stats (vel/tec/def/...) o history.\* → solo admin
- Hacerse admin a uno mismo (`isAdmin` field)
- Modificar otros jugadores, partidos, items de tienda, votos ajenos
- Borrar partidos/temporadas/items sin ser admin

Lo que **permiten** al jugador en su propio doc:

`nickname, position, photo, emoji, equipped, inventory, coins, claimedChallenges, status`

Trade-offs conocidos:

- El campo `coins` es self-writable (compras/apuestas/cajas funcionan client-side). Un usuario podría manipularlo. Mitigable con Cloud Functions más adelante.
- Cross-player coin transfer permitido (+1 exacto) para el award de MVP/Arquero.

## 5. Por qué cada pieza está donde está

- **Frontend en Vercel**: deploy automático, gratis hasta cierto tráfico, HTTPS automático, CDN global. La app es estática (SPA), no necesita servidor propio.
- **Firebase para datos**: tiempo real (snapshots), auth incluido, sin gestionar servidores ni bases. Free tier alcanza para grupos chicos.

- **PWA:** para que se sienta como app nativa en el celu (ícono en pantalla de inicio, splash, offline) sin pasar por App Store / Play Store.
- **Reglas como código:** `firestore.rules` versionado en git para tener historial y poder revisar cambios, aunque se deploya manualmente.

---

## 6. Archivos necesarios

Estos son los archivos que **no pueden faltar** para que la app corra y se deploje:

### Raíz del proyecto

- `package.json` — declara las dependencias (React, Vite, Firebase, etc.) y los scripts (`dev`, `build`).
- `package-lock.json` — snapshot exacto de versiones (lo lee Vercel).
- `vite.config.js` — config de Vite + plugin PWA (genera el service worker y el manifest).
- `index.html` — punto de entrada HTML (incluye favicon, meta tags PWA y carga `main.jsx`).
- `firestore.rules` — reglas de seguridad de la DB (versionadas en git; se publican manualmente en Firebase Console).
- `.gitignore` — qué no se sube (`node_modules`, `.env`, `dist`).

### `public/` (assets estáticos)

- `favicon.svg` — logo (tab del navegador, ícono PWA, hero, navbar).

### `src/` (código fuente)

- `main.jsx` — entrada React, monta `<App />` + registra el SW.
- `App.jsx` — decide qué pantalla mostrar (login / onboarding / app).
- `index.css` — estilos globales (fondo hexagonal, animaciones, glass, etc.).
- `firebase/config.js` — config de Firebase leído desde variables de entorno.
- `pages/` — una página por pestaña (HomePage, MatchPage, etc.).
- `components/` — reusables (PlayerCard, NavBar, EmptyState, ErrorBoundary, Toast, etc.).
- `hooks/` — capa de datos (`useAuth`, `useMatches`, `useShopItems`, `useBets`, `useMoments`, etc.).
- `utils/` — lógica pura (stats, logros, shop, teams, challenges).

### Variables de entorno (en Vercel y en `.env` local — no se suben a git)

```
VITE_FB_API_KEY=...
VITE_FB_AUTH_DOMAIN=...
VITE_FB_PROJECT_ID=...
VITE_FB_STORAGE_BUCKET=...
VITE_FB_MESSAGING_SENDER_ID=...
VITE_FB_APP_ID=...
```

### Generados automáticamente (no se editan a mano):

- `dist/` — build de producción (lo genera `npm run build`, lo sirve Vercel).
- `node_modules/` — dependencias instaladas (gitignored).

---

## 7. Nuestro flujo de trabajo

### Para hacer cualquier cambio en la app:

```
1. Editar archivos en src/ (o firestore.rules, vite.config.js, etc.)
```

```

2. git add . && git commit -m "qué cambió"
3. git push origin main
4. Vercel detecta el push automáticamente:
   - corre npm install + npm run build
   - publica dist/ a la URL pública
   - tarda ~30-60s
5. Usuarios abren la app → la PWA detecta versión nueva → auto-update
6. Si tocaste firestore.rules → ENTRAR a Firebase Console:
   Firestore Database → Rules → pegar el archivo → Publish

```

### Atajos según qué cambiás:

| Querés cambiar...      | Tocás...                                                              | Cómo se publica                                   |
|------------------------|-----------------------------------------------------------------------|---------------------------------------------------|
| UI, texto, layout      | <code>src/pages/*</code> o <code>src/components/*</code>              | git push → Vercel                                 |
| Lógica / hooks         | <code>src/hooks/*</code> o <code>src/utils/*</code>                   | git push → Vercel                                 |
| Estilos globales       | <code>src/index.css</code>                                            | git push → Vercel                                 |
| Reglas de seguridad    | <code>firestore.rules</code>                                          | <b>Manual:</b> Firebase Console → Rules → Publish |
| Defaults de la tienda  | <code>src/utils/shop.js</code><br>( <code>DEFAULT_SHOP_ITEMS</code> ) | git push → Vercel                                 |
| Items custom de tienda | Desde la UI (admin → +Agregar / =■ editar)                            | Se guarda en Firestore en vivo, sin deploy        |
| Logo / favicon         | <code>public/favicon.svg</code>                                       | git push → Vercel                                 |
| Variables de Firebase  | <code>.env</code> local y vars en Vercel                              | Re-deploy                                         |

### Reglas de oro:

- Si tocás `firestore.rules`, después tenés que **publicarlas a mano** en Firebase. Vercel no las toca.
- Los datos en Firestore (jugadores, partidos, items custom) **no se pierden** con un deploy — viven en Firebase, no en el código.
- El **commit a main** dispara el deploy. No hay paso de revisión ni branch staging configurado: lo que pushes va a producción.
- Si algo rompe la app, el **ErrorBoundary** muestra el mensaje del error en lugar de pantalla en blanco.